

Nova · Multi-Agent Trade Document Verification Pipeline

GoComet Full-Stack AI Engineer DAW · Part 1 · v1.0 · 2026-09-28 · Author: Akash · Status: Execution-ready

1 | Understanding Nova

What is Nova, and what can traditional SaaS not do?

Nova is GoComet's platform of autonomous Digital Agentic Workers (DAWs): goal-oriented agents that own an operational outcome end to end. Conventional logistics SaaS digitizes paperwork, but every decision still lands in a human queue. An operations desk handles hundreds of trade documents a day (invoices, bills of lading, packing lists), and a portal only lets people view them while they cross-check each one against purchase orders, tariff schedules and port rules. Nova's agents extract the fields, validate them against customer rules, clear the compliant majority on their own and escalate only genuine exceptions. The gap is not better OCR, which has existed for years. It is decision authority: approving a clean shipment in seconds, catching a transposed HS digit before customs submission, and drafting a supplier amendment email within minutes of ingestion. A passive record-keeping system cannot do that.

The FDE (Forward Deployed Engineer) model

International trade documentation is not standardized. It is customer-, corridor- and regulation-specific. An auto-parts importer and a fashion retailer differ in consignee legal-entity formats, approved ports, HS classification trees and Incoterm preferences. Off-the-shelf agent templates fail here because the hundreds of micro-rules live in an operations team's collective memory. GoComet therefore embeds FDEs inside customer operations. They watch real document flows, interview CG coordinators and turn that tribal knowledge into machine-readable rules: YAML configs, consignee alias dictionaries, port LOCODE rosters and weight tolerances. They also build golden evaluation sets from production edge cases and keep tuning extraction prompts and confidence thresholds. Without that discipline, an AI deployment would go live with generic rules and silently approve the first non-standard invoice it meets.

System of Outcomes vs. System of Record vs. System of Engagement

A **System of Record** stores data: a database with a UI. A **System of Engagement** adds collaboration around it (notifications, comments, status badges). Neither is responsible for what happens next. A **System of Outcomes** owns the business result. When Nova ingests an invoice it does not just store the PDF and ping someone. It extracts eight critical fields, validates each against customer rules, returns a match / mismatch / uncertain verdict, and then auto-clears the shipment, escalates with a structured breakdown, or drafts an amendment email with exact found-vs-expected values. The product is the outcome, such as a shipment clearing customs without a hold, not the document record. That changes what we measure: not documents uploaded, but shipments cleared without a human touching a field, at zero wrongful approvals.

2 | Problem Statement

Where the current validation flow breaks

CG Operators audit 80–200 trade documents a day, comparing consignee names character by character, looking up HS codes against tariff authorizations and checking weights against SOLAS VGM rules. The flow breaks at four points:

- Volume-induced fatigue errors.** The 80th invoice at 4 PM gets less vigilance than the first at 9 AM, so digit swaps and **Ltd.** vs **Inc.** slip through.
- Latency between ingestion and action.** Documents sit in a queue for hours, the supplier hears about a discrepancy late, and the vessel cut-off may pass first.
- Inconsistent escalation judgment.** Two operators can treat "Meridian Robotics Pvt Ltd" vs "Inc." differently. No deterministic, auditable rule governs the edge case.
- No feedback loop on errors.** A silently approved error becomes a customs hold costing **\$150–\$500 per container per day** (demurrage and detention), found days later and hard to trace to a field.

Four document failure modes Nova is built to catch

#	Failure mode	Example	Business impact
1	HS code digit transposition	Invoice shows 8479.05 instead of authorized 8479.50 (Industrial Robots)	Wrong duty rate; regulatory penalties at customs entry
2	Consignee legal-entity mismatch	"Meridian Robotics Ltd." instead of bonded entity "Meridian Robotics Inc."	Immediate customs bond filing rejection
3	Missing or prohibited Incoterm	Incoterm blank, or EXW when only FOB / CIF / DAP are authorized	Unresolved freight-insurance liability; legal ambiguity
4	Gross weight exceedance / unit confusion	58,400 KG against a 50,000 KG limit, or LBS with no unit tag	Container weight misdeclaration under SOLAS VGM

Operator success in the first 5 minutes. From the moment a document is ingested, the CG Operator either (1) gets a **verified auto-approval** without touching a single field, or (2) gets an **action-ready discrepancy summary** (expected vs. found) with a **pre-drafted supplier amendment email** ready for one-click dispatch, all in **under 5 minutes**.

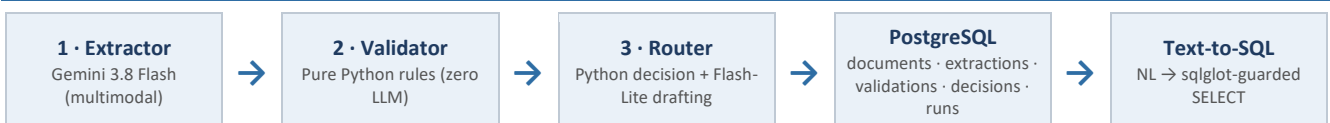
3 | Users and Jobs-to-be-Done

	Persona 1: CG Operator (Customer Ops Specialist)	Persona 2: SU (Supplier Shipping Coordinator)
Role	Import/export logistics specialist at a freight forwarder or enterprise cargo owner	Shipping coordinator at an overseas manufacturer or export depot
Does	Audits 80–200 documents a day against customer master data, clears compliant shipments for customs filing, resolves discrepancies before port cut-offs	Generates commercial invoices, packing lists and transport documents for international buyers
Primary pain	Fears silent approval errors that cause customs penalties; drowning in repetitive re-keying from fragmented PDF scans	Gets vague discrepancy queries days after vessel departure; multi-round email chains with no specific corrections
Needs	Zero false approvals; per-field confidence with source quotes; one-click amendment actions; no re-entry for clean documents	Fast, unambiguous feedback naming the failed field, found value, expected format and fix, in a single email cycle

Five testable jobs-to-be-done

1. **Ingestion and extraction.** When an SU submits a trade document (PDF or scan), I want all critical customs fields extracted with verbatim source quotes and calibrated confidence, so that I never re-key shipping data by hand.
2. **Deterministic rule validation.** When fields have been extracted, I want each one checked deterministically against declarative customer rules and marked **match**, **mismatch** or **uncertain**, so that non-compliant shipments are flagged before customs filing.
3. **Zero silent approvals.** When a field has low confidence, an ungrounded quote or ambiguous formatting, I want the document escalated to human review rather than guessed at, so that no inaccurate or hallucinated data is ever silently approved.
4. **Automated discrepancy escalation.** When discrepancies or missing requirements are found, I want an immediate expected-vs-found breakdown and a pre-drafted amendment email, so that I can resolve the issue in one click without writing the email myself.
5. **Natural-language shipment inquiries.** When auditing past decisions or investigating bottlenecks, I want to query shipment records in plain English and see the generated SQL, so that I can check compliance trends and throughput without a database engineer.

4 | Agent Architecture



Handoffs are typed Pydantic contracts: **ExtractedDoc** → **ValidationResult** → **DecisionResult** → PostgreSQL. State lives in PostgreSQL (business tables plus LangGraph checkpoints).

Agent	Responsibility	Input → Output	Trust model	Failure regime
Extractor (perception)	Read PDF/image with a multimodal LLM; return 8 fields, each with value, confidence and verbatim source_quote	PDF / image → ExtractedDoc	Can hallucinate; treated as untrusted	Ungrounded field → null
Validator (verifier)	Check every field against declarative YAML customer rules; assign match / mismatch / uncertain	ExtractedDoc + rules → ValidationResult	Deterministic Python, zero LLM; cannot hallucinate	Rule failure → mismatch / uncertain
Router (policy gate)	Decide auto_approve / human_review / amendment_request in Python; LLM only writes explanation and email draft	ValidationResult → DecisionResult	Decision deterministic; explanation generative	LLM failure → template fallback

Communication and contracts

No free-text message passing and no shared memory blobs. Every model uses `ConfigDict(extra="forbid")`, so an undeclared field fails schema validation at the boundary instead of quietly propagating bad data.

Why three agents: not one, not five

- **Why not one prompt?** (1) No self-grading: a model that extracts a consignee and then validates it will rationalize its own hallucination. The Extractor never sees customer rules and the Validator never calls an LLM. (2) No modular evals: in a monolith, tuning HS-code extraction can silently break Incoterm parsing; isolated agents get independent regression tests (80 unit tests across 13 modules). (3) Wasteful economics: perception needs multimodal pricing (\$0.75 / \$3.75 per 1M tokens) while policy drafting needs text only (\$0.075 / \$0.30).
- **Why not five?** (1) An agent boundary is only justified by a different trust model or execution mode; splitting OCR from parsing puts two agents in the same perception domain and adds no safety gate. (2) Each extra LLM hop adds an API round-trip, multiplying latency 3–5x. (3) Every boundary is another serialization point that can fail.

State persistence and crash recovery

The pipeline is a LangGraph `StateGraph` with a `PostgresSaver` checkpointer, committing a transactional snapshot after every node, keyed by `thread_id` = document UUID. After an LLM timeout, worker crash or network partition the graph reloads its last checkpoint: a document that finished extraction but crashed in validation resumes at the validator, so it is never re-extracted (no repeated token cost, no duplicate records). Resume is `graph.invoke(None, config)`.

5 | LLM and Tooling Choices

Role	Model	Cost / 1M tokens	Why
Extractor (primary)	<code>gemini-3.8-flash</code>	\$0.75 in / \$3.75 out	Recommended multimodal Flash tier; native JSON output via <code>response_schema</code> removes post-processing parse failures
Extractor (fallback)	<code>gemini-2.5-flash</code>	\$0.15 / \$0.60	Used when the primary is unavailable or a degraded scan yields fewer than 6 of 8 grounded fields; one second attempt
Router (drafting)	<code>gemini-3.5-flash-lite</code>	\$0.075 / \$0.30	Only formats structured reasoning and drafts email from validated input; needs no vision or deep reasoning
Text-to-SQL	<code>gemini-3.5-flash-lite</code>	\$0.075 / \$0.30	SQL from a known schema is pattern matching; the cheap tier is enough

Single vendor by design (Google Gemini). For a time-boxed POC, one API key, one retry/backoff implementation and one pricing table cut integration surface. All model IDs live in `.env`, so any agent can be swapped to Anthropic or OpenAI without code changes.

Component	Choice	Why
Orchestration	LangGraph <code>StateGraph</code>	Explicit typed state handoffs, built-in Postgres checkpointing, no hidden memory/chain abstractions obscuring data flow
Backend	Python + FastAPI	Pydantic models double as API schemas and agent contracts; async handling; DI for DB sessions
Database	PostgreSQL 16	Verified outputs, checkpoints and the <code>runs</code> telemetry table in one ACID system with JSON columns for raw payloads
PDF parsing	PyMuPDF / pdfplumber, text layer first	Free and deterministic, and supplies the grounding source for verbatim quote checks; vision mode is the fallback
SQL safety	<code>sqlglot</code> AST validation	Enforces read-only <code>SELECT</code> , rejects unauthorized tables/functions, auto-appends <code>LIMIT 100</code> ; no regex hacks
Frontend / deploy	React + Tailwind; <code>docker-compose</code>	Minimal UI showing real pipeline state; one <code>docker-compose up</code> boots Postgres, API and UI

Structured output vs. free text

- **Structured output (JSON schema):** Extractor (`DocumentExtractionPayload`, guarantees parseable JSON with all 8 fields), Router (`RouterLLMResponse`, keeps reasoning and email draft in separate fields) and Text-to-SQL (clean SQL extraction, no prompt-injection leakage).
- **Deliberately not LLM-driven:** the approve / review / amend decision, all validation, and the amendment-email fallback. If the LLM draft omits a discrepancy field name or found value, a deterministic template that always lists every discrepancy replaces it.

6 | Trust, Failure Handling and Evals

Stopping hallucination: code-level grounding

Prompt instructions are not a safety mechanism, so every field passes a three-step check in `grounding.py`:

1. **Source quote verification:** the quote must exist in the raw document text (3-tier match: exact, whitespace-normalized, case-insensitive normalized).
2. **Value-in-quote verification:** the value must be derivable from the quote (HS code digits appear in the quote; at least 70% of meaningful consignee tokens appear).
3. **Confidence calibration:** grounding fails → confidence `0.0` and value `null`. Grounded but syntactically invalid (e.g. HS code not `^\d{4}\.\d{2}$`) → confidence capped at `0.45`. Confidence is never inflated above the model's own report.

The invariant is enforced in the schema: a `model_validator` on `ExtractedField` sets `value = None` whenever `is_grounded == False`, so an ungrounded value cannot reach the Validator.

Zero silent approvals and low-confidence handling

`auto_approve` requires, for all 8 fields: grounded, confidence ≥ 0.85 and validator status `MATCH`, plus overall status `MATCH` and zero discrepancies. Any single violation collapses the decision to `human_review` or `amendment_request`.

- A missing Incoterm returns `value=null, confidence=0.0, status=uncertain`. The system never guesses a plausible value; an ungrounded field is nullified and marked `uncertain`.
- Vision-mode extractions with no independent text/OCR verification are capped at `0.60`, below the 0.85 threshold, which guarantees human review.
- The Validator is pure Python, so an LLM cannot hallucinate an approval through it. Measured on the 12-document golden set (7 with planted discrepancies), the false-approve rate is **0.0%**.

Stopping runaway cost, loops and retries

Control	Implementation
Retry cap	Maximum 2 model fallback attempts per document (primary → fallback → Flash-Lite)
Timeout / loop limit	<code>LLM_TIMEOUT_SECONDS = 60</code> (configurable); LangGraph recursion limit prevents infinite node loops
Cost tracking	Every LLM call logs prompt, completion and thinking tokens plus <code>cost_usd</code> to the <code>runs</code> table
Router degradation	The decision is computed in Python before any LLM call; if the Gemini call fails, a template writes the explanation
Fail-loud policy	Pipeline errors set <code>status = "FAILED"</code> and never fall through to approval

Evaluation strategy

- **Offline eval** (`evals/run_evals.py`): 12 golden documents: 3 clean (should auto-approve), 7 with planted discrepancies (should flag or amend), 1 fuzzy-consignee-alias edge case (should pass) and 1 messy scan (degraded, valid data). Measures field accuracy, quote-grounding rate, rule-validation accuracy, routing accuracy and false-approve rate.
- **Online metric: human override rate**, the share of documents where a CG operator changes the agent's routing decision. Target below 15%; persistent overrides mean the rules or thresholds need FDE tuning.
- **Automated tests:** 80 passing tests across 13 modules covering each agent contract, grounding, storage, query service, trust regressions and API.

7 | Metrics and Success Criteria

North star: Automated Safe Clearance Rate. The percentage of trade documents cleared or correctly escalated with zero manual field re-keying and zero wrongful approvals.

#	Supporting metric	Target	Measurement	Why it matters
1	False-approve rate	0.0% (non-negotiable)	Golden set with planted discrepancies	Prevents customs fines, demurrage, detention
2	Field extraction accuracy	$\geq 95\%$ clean / $\geq 85\%$ degraded	Exact match vs. ground truth	Minimizes manual re-keying
3	Source quote grounding rate	$\geq 95\%$	Automated grounding pass rate	Prevents hallucinated values
4	Human override rate	< 15%	Production tracking of CG changes	Validates operator trust

#	Supporting metric	Target	Measurement	Why it matters
5	Pipeline latency (P90)	< 15 s	Ingestion → final decision timestamp delta	Supports the 5-minute SLA
6	Avg cost per document	< \$0.015	Cumulative token cost from runs	Protects margin vs. manual work
7	Amendment single-cycle resolution	> 60%	Supplier responses needing 0 follow-ups	Measures email draft quality
8	Validator determinism	100%	Same input → same output across runs	No non-determinism in the rules engine

Measured results (Part 1 POC)

Metric	Result	Metric	Result
Field extraction accuracy	100% (12/12)	Decision routing accuracy	100%
False-approve rate	0.0% (0/7)	Avg latency (text-layer)	76.1 ms
Quote grounding rate	98.96%	Live multimodal latency	~1.2–2.2 s
Rule validation accuracy	100%	Cost per document	~\$0.00021–0.00032

Automated tests: 80 / 80 passing.

Go / No-Go for a 2-week single-customer pilot

Gate	Criterion	Threshold
MUST PASS	False-approve rate on the customer's historical document set	= 0.0%
MUST PASS	All 5 behaviours (A–E) operational end to end	All green
MUST PASS	Pipeline latency P90	< 15 seconds
SHOULD PASS	Field extraction accuracy on clean customer docs	≥ 92%
SHOULD PASS	CG operator qualitative feedback (5-minute resolution)	Positive from ≥ 2 of 3 operators
NICE TO HAVE	Human override rate in week 1	< 25% (relaxed from 15%)

If any MUST PASS gate fails, the answer is No-Go: stop, tune rules and prompts with FDE, then retry.

8 | What's Next (After Part 1 Ships)

1. **Multi-document cross-validation.** Extend the existing **doc_type** field to three-way reconciliation of Bill of Lading, Commercial Invoice and Packing List, checking consignee, weights and HS codes for consistency. Cross-document inconsistency is the largest single source of customs holds, and a correct invoice with a mismatched BOL is still a rejection waiting to happen.
2. **CG operator email workflow.** Wire the Router's drafted amendment emails into an operator-gated dispatch flow: the operator reviews, optionally edits, and sends. Nova never sends autonomously. This removes the 15–20 minutes operators spend drafting discrepancy emails and shortens the "last mile" that limits the 5-minute SLA.
3. **Confidence threshold auto-calibration.** Use accumulated **runs** data to build empirical calibration curves and tune the 0.85 threshold per field and per customer, lowering override rate without giving up zero false approvals.

Why these, in this order: cross-validation closes the highest-impact failure Part 1 cannot catch. The email workflow directly cuts time-to-resolution. Calibration comes third because it needs production data; the golden set is too small for statistical calibration. **Deprioritized on purpose:** automated IMAP watchers (convenience, not safety-critical), a multi-customer rule-management UI (FDEs can edit YAML directly during the pilot) and Langfuse (the native **runs** table covers pilot observability).